

# AI HEALTHCARE SYSTEM

## Backend Pipeline — Technical Reference Report

Methodologies • Data Sources • Technologies

| Component               | Methodology                          | Primary Dataset                          | Core Technology                    |
|-------------------------|--------------------------------------|------------------------------------------|------------------------------------|
| Disease Prediction      | Fine-tuned BERT deep learning        | 5,000 clinical notes / 20 diagnoses      | HuggingFace Transformers + PyTorch |
| Medicine Recommendation | Cosine Similarity (NLP bag-of-words) | 9,720 medicine records                   | scikit-learn + NLTK                |
| ADR Detection           | NER + ML classification + SIDER DB   | 3,107 drug reviews + 153,663 SIDER pairs | spaCy + scikit-learn + SQLite      |

Prepared by: Healthcare AI Research Division | April 2026

## Table of Contents

- 1. Executive Summary ..... 3
- 2. Backend Architecture Overview ..... 3
- 3. Component 1 — Disease Prediction from Clinical Notes ..... 4
  - 3.1 Methodology ..... 4
  - 3.2 Data Source ..... 4
  - 3.3 Full Backend Pipeline (8 Steps) ..... 5
  - 3.4 Technologies & Libraries ..... 6
  - 3.5 API Design & Outputs ..... 6
  - 3.6 Performance Metrics ..... 7
- 4. Component 2 — Medicine Recommendation System ..... 7
  - 4.1 Methodology ..... 7
  - 4.2 Data Source ..... 8
  - 4.3 Full Backend Pipeline (7 Steps) ..... 8
  - 4.4 Technologies & Libraries ..... 9
  - 4.5 API Design & Outputs ..... 9
  - 4.6 Performance Metrics ..... 10
- 5. Component 3 — ADR Detection System ..... 10

- 5.1 Methodology ..... 10
- 5.2 Data Sources ..... 11
- 5.3 Full Backend Pipeline (8 Steps) ..... 11
- 5.4 Technologies & Libraries ..... 13
- 5.5 API Design & Outputs ..... 13
- 5.6 Performance Metrics ..... 14
- 6. Integrated Backend Architecture ..... 14
- 7. Deployment & Infrastructure ..... 15
- 8. Limitations & Future Work ..... 15
- 9. Conclusion ..... 16

# 1. Executive Summary

This document is the authoritative backend technical reference for a three-component AI healthcare platform. It covers every stage of the data processing and model-building pipeline for each component, detailing the methodology (how the problem is solved), data sources (what information is used), and technologies (which tools and frameworks are employed). The platform is designed to be deployed as a set of REST API microservices that can be integrated into hospital information systems, pharmacy management platforms, or clinical decision-support tools.

## Who This Report Is For

This report is written for backend engineers, data scientists, project managers, and stakeholders who need to understand how the AI models work — without necessarily having prior knowledge of machine learning or clinical informatics. Technical terms are explained in plain language throughout.

The three components and their core purpose:

- Component 1 — Disease Prediction: Reads a doctor's written clinical notes and predicts the patient's most likely diagnosis from 20 possible conditions.
- Component 2 — Medicine Recommendation: Given one medicine name, returns five clinically similar alternatives based on medical descriptions and therapeutic use.
- Component 3 — ADR Detection: Reads a patient's written drug review, identifies the drug and any symptoms mentioned, assesses the severity of any adverse reaction, and cross-checks findings against a verified medical database.

# 2. Backend Architecture Overview

The backend is structured as three independent microservices, each with its own data pipeline, trained model, and API endpoint. All three services can be called independently or chained together for a complete clinical workflow.

| Microservice | Endpoint (example) | Model Format | Average |
|--------------|--------------------|--------------|---------|
|--------------|--------------------|--------------|---------|

|                         |                                 |                                       | Latency                   |
|-------------------------|---------------------------------|---------------------------------------|---------------------------|
| Disease Prediction      | POST /api/v1/predict-disease    | HuggingFace BERT (.bin / config.json) | ~300ms (CPU), ~80ms (GPU) |
| Medicine Recommendation | POST /api/v1/recommend-medicine | scikit-learn cosine matrix (.pkl)     | ~20ms (in-memory lookup)  |
| ADR Detection           | POST /api/v1/detect-adr         | spaCy NER + sklearn .pkl + SQLite .db | ~150ms                    |

### Shared Infrastructure

All three services are built in Python 3.12, served via a FastAPI or Flask REST layer, containerised with Docker, and managed via a CI/CD pipeline. Models are loaded once at startup and kept in memory for low-latency inference. Logging, rate-limiting, and authentication are handled by an API gateway layer (e.g., NGINX or AWS API Gateway).

## 3. Component 1 — Disease Prediction from Clinical Notes

### 3.1 Methodology

The disease prediction component uses Transfer Learning with BERT (Bidirectional Encoder Representations from Transformers). Transfer learning means starting with a model that already understands the English language — trained by Google on billions of text documents — and then teaching it the specific task of medical diagnosis classification. This approach achieves high accuracy even with a relatively small medical dataset (5,000 records), because the model does not need to learn grammar and vocabulary from scratch.

### Why BERT?

BERT reads text bidirectionally — meaning it considers words to the left AND right of each word simultaneously. This is critical for medical notes where word order and context determine meaning. For example, 'patient has no chest pain' and 'patient has chest pain' mean the opposite, and BERT captures this distinction. Traditional word-frequency methods cannot.

The methodology in three phases:

1. Pre-processing Phase — Clinical notes are cleaned (numbers removed, punctuation stripped, filler words eliminated) and converted to a numerical format BERT can read.
2. Fine-tuning Phase — The pre-trained BERT model is augmented with a new classification layer that has 20 outputs (one per diagnosis). It is then trained on the 4,000 medical examples, adjusting its internal weights to specialise in diagnosis prediction.
3. Inference Phase — When a new patient note arrives at the API, it is tokenised, passed through the fine-tuned model, and the output with the highest confidence score is converted back to the disease name and returned.

### 3.2 Data Source

**Dataset: clinical\_notes\_diagnosis\_prediction\_5000.csv**

Records: 5,000 anonymised patient clinical notes  
Columns: Clinical Notes (free-text narrative) | Diagnosis (confirmed label)  
Diagnosis classes: 20 unique conditions  
Class balance: Roughly equal — 241 to 284 records per condition  
Example note: 'A 50-year-old male presents for a follow-up visit. He has a history of hypertension but has not been compliant with his medications. Blood pressure readings are consistently elevated at 160/100 mmHg...'

| Diagnosis Category                                                      | Approx. Record Count |
|-------------------------------------------------------------------------|----------------------|
| Peptic Ulcer Disease                                                    | 284                  |
| Type 2 Diabetes Mellitus                                                | 283                  |
| Acute Myocardial Infarction                                             | 272                  |
| Chronic Obstructive Pulmonary Disease                                   | 269                  |
| Cerebrovascular Accident (Stroke)                                       | 263                  |
| Deep Vein Thrombosis                                                    | 260                  |
| Community-Acquired Pneumonia                                            | 251                  |
| Chronic Kidney Disease                                                  | 251                  |
| Septic Shock, Rheumatoid Arthritis, Congestive Heart Failure + 9 others | 241–250 each         |

### 3.3 Full Backend Pipeline (8 Steps)

|           |                                                                                                                                                                                                                                                                                                                                                                                           |
|-----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| STEP<br>1 | <b>Data Ingestion &amp; Validation</b>                                                                                                                                                                                                                                                                                                                                                    |
|           | The CSV file is loaded into a pandas DataFrame. Integrity checks confirm zero missing values, correct column names, and the expected number of rows. Any anomalous records (empty notes, unknown diagnosis labels) are flagged and removed.                                                                                                                                               |
|           | <b>INPUT:</b> Raw CSV: 5,000 rows × 2 columns (Clinical Notes, Diagnosis)<br><b>OUTPUT:</b> Clean validated DataFrame; data quality report                                                                                                                                                                                                                                                |
|           | <b>TECHNOLOGY:</b> Python, pandas                                                                                                                                                                                                                                                                                                                                                         |
| STEP<br>2 | <b>Text Preprocessing</b>                                                                                                                                                                                                                                                                                                                                                                 |
|           | Each clinical note undergoes a five-step cleaning pipeline: (1) convert to lowercase, (2) remove numeric digits, (3) strip punctuation and non-alphabetical characters, (4) collapse multiple spaces to single spaces, (5) remove English stopwords (common words like 'the', 'a', 'is' that carry no diagnostic meaning). This reduces each note to its medically meaningful vocabulary. |

|        |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|--------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|        | <p><b>INPUT:</b> Raw note text (with numbers, punctuation, stopwords)</p> <p><b>OUTPUT:</b> Cleaned note text (e.g. 'yearold male presents heartburn regurgitation sour taste...')</p> <p><b>TECHNOLOGY:</b> Python re (regex), NLTK (stopwords corpus)</p>                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| STEP 3 | <p><b>Label Encoding</b></p> <p>Machine learning models require numbers, not text. The 20 diagnosis strings are converted to integer codes 0–19 using a deterministic Label Encoder. The encoder mapping is saved to disk so the numeric output of the model can be converted back to a human-readable disease name at inference time.</p> <p><b>INPUT:</b> List of 20 unique diagnosis strings</p> <p><b>OUTPUT:</b> Integer codes 0–19; label_encoder.pkl saved to disk</p> <p><b>TECHNOLOGY:</b> scikit-learn LabelEncoder, Python pickle</p>                                                                                                                                                   |
| STEP 4 | <p><b>Train / Validation Split</b></p> <p>The 5,000 records are shuffled and split: 4,000 (80%) are used for model training; 1,000 (20%) are reserved as a held-out test set that the model will never see during training. This is the standard way to measure how well a model will perform on truly unseen data.</p> <p><b>INPUT:</b> 5,000 labelled records</p> <p><b>OUTPUT:</b> train_texts (4,000), test_texts (1,000), train_labels (4,000), test_labels (1,000)</p> <p><b>TECHNOLOGY:</b> scikit-learn train_test_split</p>                                                                                                                                                               |
| STEP 5 | <p><b>BERT Tokenisation</b></p> <p>Each cleaned note is fed through the BERT tokeniser. The tokeniser breaks each note into sub-word tokens (e.g. 'hypertension' → ['hyper', '##tension']), converts them to vocabulary IDs, and pads or truncates sequences to exactly 512 tokens. An attention mask is also generated — a binary array where 1 = real token, 0 = padding. This is required by BERT to avoid treating padding as meaningful content.</p> <p><b>INPUT:</b> Cleaned note text strings (train and test sets)</p> <p><b>OUTPUT:</b> input_ids tensors (token IDs), attention_mask tensors; max length 512</p> <p><b>TECHNOLOGY:</b> HuggingFace BertTokenizer (bert-base-uncased)</p> |
| STEP 6 | <p><b>BERT Model Fine-Tuning</b></p> <p>The pre-trained BERT model (110 million parameters) is loaded with an added classification head — a final linear layer with 20 output neurons. Training runs for 5 epochs with: learning rate 2e-5 (very small, to preserve pre-trained knowledge), batch size 16, weight decay 0.01 (regularisation to prevent overfitting), linear learning rate warm-up for the first 500 steps. The best checkpoint (lowest validation loss) is retained automatically.</p> <p><b>INPUT:</b> Tokenised training dataset (4,000 samples); pre-trained bert-base-</p>                                                                                                    |

|  |                                                                                                            |
|--|------------------------------------------------------------------------------------------------------------|
|  | uncased weights                                                                                            |
|  | <b>OUTPUT:</b> Fine-tuned model weights; checkpoints per epoch; training logs                              |
|  | <b>TECHNOLOGY:</b> HuggingFace BertForSequenceClassification, Trainer, TrainingArguments, datasets library |

|           |                                                                                                                                                                                                                                                                                                                                          |
|-----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| STEP<br>7 | <b>Evaluation</b>                                                                                                                                                                                                                                                                                                                        |
|           | The fine-tuned model makes predictions on the 1,000 held-out test records. For each test note, the model outputs 20 probability scores; the highest is taken as the predicted diagnosis. Results are compiled into a classification report (precision, recall, F1-score per disease) and visualised as a 20×20 confusion matrix heatmap. |
|           | <b>INPUT:</b> Fine-tuned model + 1,000 unseen test samples                                                                                                                                                                                                                                                                               |
|           | <b>OUTPUT:</b> Per-class precision / recall / F1 report; confusion matrix PNG                                                                                                                                                                                                                                                            |
|           | <b>TECHNOLOGY:</b> scikit-learn classification_report, confusion_matrix; seaborn heatmap                                                                                                                                                                                                                                                 |

|           |                                                                                                                                                                                                                                                                                                                                             |
|-----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| STEP<br>8 | <b>Model Serialisation &amp; API Packaging</b>                                                                                                                                                                                                                                                                                              |
|           | The trained model and tokeniser are saved using HuggingFace's native format (save_pretrained). The label encoder is saved with pickle. A predict_disease() function wraps the full inference pipeline: tokenise → forward pass → argmax → inverse-transform label → return disease name. This function is exposed via the backend REST API. |
|           | <b>INPUT:</b> Trained model weights, tokeniser vocabulary, label encoder                                                                                                                                                                                                                                                                    |
|           | <b>OUTPUT:</b> patient_model/ directory, label_encoder.pkl, patient_model.zip, predict_disease() API handler                                                                                                                                                                                                                                |
|           | <b>TECHNOLOGY:</b> HuggingFace save_pretrained, Python pickle, shutil (zip)                                                                                                                                                                                                                                                                 |

## | 3.4 Technologies & Libraries

| Library / Tool           | Version     | Purpose                                               |
|--------------------------|-------------|-------------------------------------------------------|
| Python                   | 3.12        | Core language                                         |
| pandas                   | 2.x         | Data loading, cleaning, splitting                     |
| NLTK                     | 3.x         | Stopwords corpus for text cleaning                    |
| HuggingFace Transformers | 4.x         | BertTokenizer, BertForSequenceClassification, Trainer |
| bert-base-uncased        | Pre-trained | Foundation language model (110M parameters)           |
| HuggingFace datasets     | 4.0.0       | PyTorch-compatible dataset format for Trainer         |
| PyTorch                  | 2.x         | Neural network computation engine                     |
| scikit-learn             | 1.x         | LabelEncoder, train_test_split, evaluation metrics    |
| seaborn / matplotlib     | Standard    | Confusion matrix visualisation                        |
| pickle                   | Built-in    | Saving label encoder                                  |
| shutil                   | Built-in    | Zipping model directory for download                  |

## 3.5 API Design & Outputs

### POST /api/v1/predict-disease

Request body (JSON): { "clinical\_note": "Patient presents with severe chest pain, shortness of breath..." }

Response (JSON): { "predicted\_diagnosis": "Acute Myocardial Infarction", "confidence": 0.94, "top\_3": [{"diagnosis": "Acute Myocardial Infarction", "score": 0.94}, ...] }

Startup behaviour: Model and tokeniser loaded from patient\_model/ directory into memory at server start.

Error handling: Returns HTTP 422 if note is empty or too short (<10 words); HTTP 503 if model not loaded.

## 3.6 Performance Metrics

**5,000**

Training records

**20**

Disease classes

**5**

Training epochs

**1.00**

Test F1-Score

### Interpretation of Results

The model achieved a perfect F1-score of 1.00 across all 20 disease categories on the held-out test set. This means every test note was correctly classified. This result reflects the structured nature of the dataset (clinical notes for the same condition follow similar patterns). Before live deployment, validation on independently collected, noisier real-world hospital EHR data is strongly recommended.

## 4. Component 2 — Medicine Recommendation System

### 4.1 Methodology

The Medicine Recommendation system uses Content-Based Filtering — the same approach Netflix uses to recommend movies based on what you have already watched. Instead of user behaviour, it uses the medical content of each drug: what conditions it treats, and how its effects are described. Two medicines are considered 'similar' if their medical descriptions share many of the same key words after normalisation.

The technical mechanism is Cosine Similarity on a Bag-of-Words matrix. Each medicine is represented as a vector of word-frequency counts across a shared 806-word medical vocabulary. The cosine of the angle between two vectors measures how similar they are — a score of 1.0 means identical, 0.0 means completely different. This approach is computationally lightweight, fully interpretable, and produces intuitively correct results for medicines targeting the same conditions.

Why Cosine Similarity Instead of Simple Keyword Matching?

Simple keyword matching would only find medicines that share exact word matches. Cosine similarity works across the entire vocabulary simultaneously, weighing multiple overlapping keywords together. Two medicines both described as 'treating moderate acne vulgaris in adolescents' will score high even if they use slightly different phrasing, because their overall word-frequency vectors will be similar.

4.2 Data Source

Dataset: medicine.csv

Records: 9,720 medicine entries  
Columns: index | Drug\_Name | Reason (condition treated) | Description (plain-English drug description)  
Data quality: Zero missing values, zero duplicate records  
Description length: Average 73 characters, range 7–260 characters  
Condition coverage: Includes Acne, Diabetes, Hypertension, Wound, ADHD, Alzheimer's, HIV/AIDS, and hundreds more

| Column      | Type | Example                                         | Backend Use                                                        |
|-------------|------|-------------------------------------------------|--------------------------------------------------------------------|
| Drug_Name   | Text | ACGEL CL NANO Gel 15gm                          | Lookup key for recommendation queries; returned in API response    |
| Reason      | Text | Acne                                            | Merged into tag vector; provides condition-level similarity signal |
| Description | Text | It is used to treat acne vulgaris in people 12+ | Primary text for similarity computation after stemming             |

4.3 Full Backend Pipeline (7 Steps)

| STEP<br>1 | Data Ingestion & EDA                                                                                                                                                                                                                                                                                                                                       |
|-----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|           | The medicine CSV is loaded and validated. Exploratory analysis generates: a bar chart of the 10 most represented medical conditions, a histogram of description lengths, and a word cloud of the most frequent medical terms across all descriptions. No missing values or duplicates are found.                                                           |
|           | <b>INPUT:</b> medicine.csv (9,720 rows × 4 columns)<br><b>OUTPUT:</b> Validated DataFrame; 3 EDA visualisation charts                                                                                                                                                                                                                                      |
|           | <b>TECHNOLOGY:</b> Python, pandas, matplotlib, seaborn, wordcloud                                                                                                                                                                                                                                                                                          |
| STEP<br>2 | Tag Construction                                                                                                                                                                                                                                                                                                                                           |
|           | The Description and Reason for each medicine are merged into a single 'tag' string. The description is split into individual words, whitespace is removed from each token, and the condition label is appended. The result is a single text field that captures everything medically relevant about the drug. All tags are then lowercased for uniformity. |

**INPUT:** Description (list of words) + Reason (list of words) per medicine  
**OUTPUT:** Single lowercase tag string per medicine: e.g. 'mild moderate acne spots acne'

**TECHNOLOGY:** Python `str.split()`, `str.join()`, `str.lower()`, list concatenation

### Word Stemming

STEP  
3

Each word in every tag is reduced to its grammatical root (its 'stem') using the Porter Stemmer algorithm. For example: 'treating' → 'treat', 'infections' → 'infect', 'prescribed' → 'prescrib'. This ensures that 'treat', 'treating', 'treated', and 'treatment' are all counted as the same word, improving similarity matching across medicines with slightly different phrasing.

**INPUT:** Lowercase tag strings per medicine

**OUTPUT:** Stemmed tag strings where all word variants are normalised to their root form

**TECHNOLOGY:** NLTK PorterStemmer

### Bag-of-Words Vectorisation

STEP  
4

A Count Vectoriser builds a shared vocabulary from all 9,720 stemmed tags. Common English words (stopwords) are excluded. The vocabulary is capped at 5,000 features (only the most frequent medically meaningful terms are retained). Each medicine's tag is then converted to a vector of integers representing how many times each vocabulary word appears. The result is a  $9,720 \times 806$  sparse matrix.

**INPUT:** 9,720 stemmed tag strings

**OUTPUT:**  $9,720 \times 806$  integer matrix; 806-word medical vocabulary

**TECHNOLOGY:** scikit-learn `CountVectorizer` (`max_features=5000`, `stop_words='english'`)

### Cosine Similarity Matrix Computation

STEP  
5

The cosine similarity between every pair of medicine vectors is computed. For two medicine vectors A and B, the similarity score =  $(A \cdot B) / (|A| \times |B|)$ . This gives a  $9,720 \times 9,720$  symmetric matrix where each cell [i, j] represents how similar medicine i is to medicine j. Higher values mean more similar; a medicine compared with itself scores exactly 1.0.

**INPUT:**  $9,720 \times 806$  integer matrix

**OUTPUT:**  $9,720 \times 9,720$  float similarity matrix; values in range [0.0, 1.0]

**TECHNOLOGY:** scikit-learn `cosine_similarity`

### Recommendation Function

STEP  
6

The `recommend(medicine_name)` function implements the query logic: (1) look up the medicine's integer index in the DataFrame, (2) retrieve that row from the similarity matrix (9,720 similarity scores), (3) sort scores descending, (4) return the top 5 entries excluding the query medicine itself. If the medicine name is not found, a

descriptive error message is returned. The function is wrapped in a try/except IndexError handler.

**INPUT:** Medicine name (string) + precomputed  $9,720 \times 9,720$  similarity matrix in memory

**OUTPUT:** List of 5 recommended Drug\_Names with their similarity scores

**TECHNOLOGY:** Python sorted(), enumerate(), pandas .iloc[], try/except

STEP  
7

## Serialisation & Deployment Packaging

The processed medicines DataFrame is serialised to medicine\_dict.pkl. The similarity matrix is compressed from 64-bit to 16-bit floating point (float16), reducing storage from ~800 MB to ~200 MB with negligible precision loss. Both files are saved via pickle. The cleaned data is also exported as new\_data.csv for audit and retraining purposes.

**INPUT:** pandas DataFrame + float64  $9,720 \times 9,720$  similarity matrix

**OUTPUT:** medicine\_dict.pkl (medicine data), model.pkl (float16 compressed matrix), new\_data.csv

**TECHNOLOGY:** Python pickle, NumPy astype('float16')

## | 4.4 Technologies & Libraries

| Library / Tool | Version  | Purpose                                           |
|----------------|----------|---------------------------------------------------|
| Python         | 3.12     | Core language                                     |
| pandas         | 2.x      | Data loading, tag construction, DataFrame export  |
| numpy          | 1.x      | Array operations, float16 compression             |
| NLTK           | 3.x      | PorterStemmer for word root normalisation         |
| scikit-learn   | 1.x      | CountVectorizer (bag-of-words), cosine_similarity |
| matplotlib     | Standard | EDA bar charts and histograms                     |
| seaborn        | Standard | Count plot for condition distribution             |
| wordcloud      | Standard | Common keyword visualisation                      |
| pickle         | Built-in | Serialising model artefacts to .pkl files         |

## | 4.5 API Design & Outputs

### POST /api/v1/recommend-medicine

Request body (JSON): { "medicine\_name": "ACGEL CL NANO Gel 15gm" }

Response (JSON): { "query": "ACGEL CL NANO Gel 15gm", "recommendations": [ { "rank": 1, "name": "ACGEL NANO Gel 15gm", "similarity": 0.98 }, { "rank": 2, "name": "Acnehit Gel 15gm", "similarity": 0.95 }, ... ] }

Startup behaviour: medicine\_dict.pkl and model.pkl loaded into memory once at server start.

Error handling: Returns HTTP 404 with message 'Medicine not found. Check spelling.' if name

not in dictionary.

## 4.6 Performance Metrics

9,720

Medicines indexed

806

Vocabulary size

5

Recommendations returned

~20ms

Query response time

### Validated Example

Input: 'ACGEL CL NANO Gel 15gm'

Output: ACGEL NANO Gel 15gm | Acnehit Gel 15gm | Acnelak Soap 75gm | Acnetor AD 1% Ointment 15gm | Acnetor AD Cream 15gm

Clinical validation: All five recommendations are acne-treatment products — clinically coherent and therapeutically relevant alternatives.

## 5. Component 3 — ADR Detection System

### 5.1 Methodology

The ADR Detection System uses a multi-stage hybrid pipeline combining three distinct AI and database techniques:

| Stage                         | Technique                                  | What It Does                                                                                           |
|-------------------------------|--------------------------------------------|--------------------------------------------------------------------------------------------------------|
| Stage 1 — Text Extraction     | Named Entity Recognition (NER) using spaCy | Identifies and labels drug names and symptoms within free-form patient review text                     |
| Stage 2 — Risk Classification | TF-IDF + Logistic Regression               | Reads the full review context and predicts whether it represents a moderate-to-severe adverse reaction |
| Stage 3 — Database Validation | SQLite query against SIDER 4.1             | Cross-checks each detected drug+symptom pair against 153,663 verified FDA/EMA drug safety records      |

### Why a Hybrid Pipeline?

No single technique solves the full ADR detection problem. NER alone identifies what is mentioned but cannot assess severity. A classifier alone cannot tell you which specific drug caused which specific symptom. Database lookup alone cannot handle new or unusual symptom descriptions. The three-stage hybrid addresses all dimensions simultaneously.

### 5.2 Data Sources

| Dataset                      | Source                                | Size               | Role                                        |
|------------------------------|---------------------------------------|--------------------|---------------------------------------------|
| drugLibTrain_raw.csv         | Drug library patient reviews          | 3,107 reviews      | NER training data + ADR classifier training |
| SIDER 4.1 side-effect-terms  | github.com/dhimmel/SIDER4             | TSV reference file | Side-effect term dictionary                 |
| SIDER 4.1 side-effects pairs | github.com/dhimmel/SIDER4             | TSV pairs file     | Drug/side-effect ground-truth pairs         |
| DrugBank drug names          | github.com/dhimmel/drugbank           | TSV name mapping   | Maps DrugBank IDs to official drug names    |
| adr_database.db (generated)  | Merged SIDER output — local SQLite DB | 153,663 pairs      | Runtime validation of detected ADR signals  |

About SIDER 4.1

SIDER (Side Effect Resource) is a publicly available database compiled from official drug label information filed with the US Food and Drug Administration (FDA) and the European Medicines Agency (EMA). It contains 153,663 verified drug/side-effect pairs, making it the gold standard for distinguishing known adverse reactions from novel or coincidental symptom reports.

5.3 Full Backend Pipeline (8 Steps)

|           |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|-----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| STEP<br>1 | <b>Data Loading &amp; Exploratory Analysis</b>                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|           | The drug review CSV is loaded. Key statistics are computed: 3,107 total reviews covering 502 unique drugs across 1,426 medical conditions. Missing values are catalogued (side-effects review: 2.41%, benefits review: 0.58%). Six EDA charts are produced: severity distribution, rating distribution, effectiveness distribution, side-effect word cloud, top-drug frequency chart, and review-length histograms.                                                                       |
|           | <b>INPUT:</b> drugLibTrain_raw.csv (3,107 rows × 9 columns)<br><b>OUTPUT:</b> Statistical overview; 6 EDA visualisation charts; missing value catalogue                                                                                                                                                                                                                                                                                                                                   |
|           | <b>TECHNOLOGY:</b> Python, pandas, matplotlib, seaborn, wordcloud                                                                                                                                                                                                                                                                                                                                                                                                                         |
| STEP<br>2 | <b>Text Cleaning &amp; Binary ADR Labelling</b>                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|           | All text review columns are cleaned: URLs removed, special characters stripped, text lowercased and whitespace normalised. Missing reviews are filled with 'none'. The 5-level severity scale is collapsed to a binary target variable: 'Moderate', 'Severe', or 'Extremely Severe' side effects → ADR Positive (label=1); 'No Side Effects' or 'Mild' → ADR Negative (label=0). A combined input text is built per record in the format 'drug: [name] cond: [condition] review: [text]'. |
|           | <b>INPUT:</b> 3,107 raw review records; 5-level sideEffects severity column<br><b>OUTPUT:</b> Clean binary labels: 1,949 Negative (62.7%), 1,158 Positive (37.3%); ml_input_text column                                                                                                                                                                                                                                                                                                   |
|           | <b>TECHNOLOGY:</b> Python re (regex), pandas string methods (.fillna, .str.lower, .apply)                                                                                                                                                                                                                                                                                                                                                                                                 |

|                              |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <div>STEP</div> <div>3</div> | <h3>NER Training Data Construction &amp; Augmentation</h3> <p>For each drug review, an automated annotation pipeline identifies: (a) drug name occurrences in the text (pattern-matched against the drug name column) and (b) symptom occurrences (regex-matched against a curated list of ~50 symptom root words including pain, nausea, dizziness, rash, headache, fatigue, etc.). Overlapping annotations are resolved by keeping the longest span. Three augmentation strategies expand the dataset: synonym substitution, sentence re-ordering, and negation injection. Total samples grow from 3,107 to 12,428.</p> <p><b>INPUT:</b> 3,107 cleaned review texts + drug names + symptom root-word dictionary</p> <p><b>OUTPUT:</b> 12,428 spaCy NER training examples, each annotated with DRUG and SYMPTOM entity spans</p> <p><b>TECHNOLOGY:</b> Python re (regex), spaCy training format (text, {'entities': [(start, end, label)]}), custom augmentation functions</p> |
| <div>STEP</div> <div>4</div> | <h3>spaCy NER Model Training</h3> <p>A blank English spaCy NLP pipeline is initialised and a NER component is added. The two entity labels DRUG and SYMPTOM are registered. Training runs for 35 epochs using mini-batch gradient descent with compounding batch sizes (starting at 4, growing to 16). A dropout rate of 0.35 prevents overfitting. The training/validation split is 85%/15%. Loss is logged every 5 epochs: epoch 5 → 680.8, epoch 35 → 86.9 (87% reduction). The model is saved to adr_ner_model/.</p> <p><b>INPUT:</b> 12,428 NER training examples; spaCy blank('en') pipeline</p> <p><b>OUTPUT:</b> Trained NER model; loss curve plot; model saved to adr_ner_model/</p> <p><b>TECHNOLOGY:</b> spaCy (blank, add_pipe('ner'), minibatch, compounding, Example, optimizer)</p>                                                                                                                                                                             |
| <div>STEP</div> <div>5</div> | <h3>Hybrid NER Enhancement — PhraseMatcher Override</h3> <p>The pure NER model can miss drug names that were rare or absent in the training data. A PhraseMatcher component is added as a post-processing step: it maintains a whitelist of all 502 known drug names from the dataset, scans each document for exact matches (case-insensitive), and forcibly labels any matches as DRUG entities — overriding the NER model if needed. The final model (nlp_final) combines both layers and is saved as adr_ner_final_certified/.</p> <p><b>INPUT:</b> Trained NER model + list of 502 known drug names from the dataset</p> <p><b>OUTPUT:</b> Hybrid NER model combining learned + rule-based entity detection; saved to adr_ner_final_certified/</p> <p><b>TECHNOLOGY:</b> spaCy (PhraseMatcher, @Language.component decorator, Span, filter_spans)</p>                                                                                                                      |
| <div>STEP</div> <div>6</div> | <h3>ML ADR Risk Classifier Training &amp; Selection</h3> <p>Two scikit-learn classification pipelines are trained: (1) Logistic Regression with TF-IDF (max 5,000 features, 1+2-gram, sublinear TF scaling, class_weight='balanced'), (2) Random Forest with the same TF-IDF input. Both are evaluated on the 20% held-</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |

out test set (622 reviews). Logistic Regression achieves AUC 0.852 vs Random Forest AUC 0.808. Logistic Regression is selected as the production model and saved via joblib.

**INPUT:** 3,107 ml\_input\_text strings + binary ADR labels; 80/20 stratified split  
**OUTPUT:** Two trained pipelines; comparison report; selected model saved as adr\_classifier\_model.pkl

**TECHNOLOGY:** scikit-learn (TfidfVectorizer, LogisticRegression, RandomForestClassifier, Pipeline, roc\_auc\_score), joblib

STEP

7

### SIDER 4.1 Database Integration

Three public TSV files are fetched from GitHub: side-effect term definitions, DrugBank drug-name mappings, and SIDER drug/side-effect pairs. These are merged via shared identifiers (UMLS CUI codes and DrugBank IDs) into a single reference table of 153,663 drug/side-effect pairs. The merged data is loaded into a local SQLite database (adr\_database.db) for fast SQL lookup at inference time. Fuzzy string matching (RapidFuzz, threshold  $\geq 90\%$ ) maps 78 of 502 dataset drug names to their official SIDER names.

**INPUT:** 3 TSV files from GitHub (SIDER4 + DrugBank repos)

**OUTPUT:** adr\_database.db (SQLite, 153,663 rows),  
certified\_drug\_mappings.csv (78 verified drug-name bridges)

**TECHNOLOGY:** pandas, sqlite3, rapidfuzz (process.extractOne, fuzz.ratio)

STEP

8

### Clinical Audit Engine — Inference Class

The ClinicalAuditEngine class unifies all components. Its analyze(text, drug\_name, condition) method: (1) formats the input as 'drug: [name] cond: [condition] review: [text]', (2) calls the Logistic Regression classifier to get ADR probability and verdict, (3) runs the hybrid NER model to extract entity spans, (4) queries SQLite for each detected drug+symptom pair. The final response bundles verdict, confidence score, extracted entities, and SIDER verification status.

**INPUT:** Free-form patient review text + optional drug name and condition

**OUTPUT:** Structured audit report: { verdict, adr\_probability, entities [(drug/symptom spans)], sider\_verified: true/false }

**TECHNOLOGY:** ClinicalAuditEngine class (spaCy + scikit-learn pipeline + SQLite sqlite3.connect())

## | 5.4 Technologies & Libraries

| Library / Tool | Version  | Purpose                                                |
|----------------|----------|--------------------------------------------------------|
| Python         | 3.12     | Core language                                          |
| pandas         | 2.x      | Data loading, cleaning, SIDER merging                  |
| numpy          | Standard | Numerical operations                                   |
| spaCy          | 3.x      | NER model training, PhraseMatcher, pipeline management |

|                      |          |                                                                      |
|----------------------|----------|----------------------------------------------------------------------|
| en_core_web_sm       | spaCy    | Base English vocabulary and tokeniser for NER pipeline               |
| scikit-learn         | 1.x      | TF-IDF vectorisation, Logistic Regression, Random Forest, evaluation |
| rapidfuzz            | Standard | Fuzzy string matching for SIDER drug name mapping                    |
| sqlite3              | Built-in | Local database for SIDER ADR reference pairs                         |
| joblib               | Standard | Saving and loading scikit-learn pipelines                            |
| matplotlib / seaborn | Standard | EDA charts, confusion matrices                                       |
| wordcloud            | Standard | Text frequency visualisation                                         |
| re (regex)           | Built-in | Text cleaning and entity pattern annotation                          |

## | 5.5 API Design & Outputs

### POST /api/v1/detect-adr

Request body: { "review\_text": "I started taking enalapril last week. By day three I had terrible dizziness.", "drug\_name": "enalapril", "condition": "hypertension" }

Response: { "verdict": "HIGH RISK (ADR)", "adr\_probability": 0.592, "risk\_level": "HIGH", "entities": [ { "text": "enalapril", "type": "DRUG" }, { "text": "dizziness", "type": "SYMPTOM" } ], "sider\_verification": { "enalapril + dizziness": "CONFIRMED — Known ADR in SIDER 4.1" } }

Startup: NER model, classifier pipeline, and SQLite connection all initialised at server start.

Error handling: HTTP 400 if review text is empty; HTTP 404 if drug not found in SIDER; HTTP 503 if any model fails to load.

## | 5.6 Performance Metrics

|               |              |                |                     |
|---------------|--------------|----------------|---------------------|
| 99.5%         | 99.5%        | 85.2%          | 153K                |
| NER Precision | NER F1-Score | Classifier AUC | SIDER Pairs Indexed |

| Metric                        | Logistic Regression | Random Forest  | Notes                         |
|-------------------------------|---------------------|----------------|-------------------------------|
| AUC                           | 0.8522              | 0.8083         | Higher is better; LR selected |
| ADR cases identified          | 175/232 (75.4%)     | ~161/232 (69%) | Critical for patient safety   |
| False negatives (missed ADRs) | 57                  | ~71            | Lower is better               |
| Safe/Mild precision           | 0.85                | 0.82           | —                             |
| ADR precision                 | 0.71                | 0.66           | —                             |
| Overall accuracy              | 79%                 | ~75%           | —                             |

## 6. Integrated Backend Architecture

The three components are designed as independent microservices but can be chained into a unified clinical workflow. The recommended integration sequence is:

| Step | API Call                 | Input                            | Output                                  | Clinical Purpose                        |
|------|--------------------------|----------------------------------|-----------------------------------------|-----------------------------------------|
| 1    | POST /predict-disease    | Doctor's clinical note text      | Predicted diagnosis (e.g. Hypertension) | Fast initial triage support             |
| 2    | POST /recommend-medicine | Medicine name for that diagnosis | Top-5 alternative medicines             | Drug selection and substitution         |
| 3    | POST /detect-adr         | Patient review of that medicine  | ADR risk verdict + SIDER verification   | Safety monitoring and pharmacovigilance |

### Shared Backend Infrastructure

Language: Python 3.12  
API Framework: FastAPI (recommended) or Flask — provides automatic OpenAPI/Swagger documentation  
Container: Docker — each microservice in its own container; orchestrated with Docker Compose or Kubernetes  
Model Storage: Local filesystem (models loaded at startup) or cloud object storage (AWS S3 / GCP GCS) for large BERT weights  
Database: SQLite (development) or PostgreSQL (production) for the SIDER ADR reference store  
Authentication: JWT tokens via API gateway  
Logging: Structured JSON logs (python-json-logger); request tracing with correlation IDs  
CI/CD: GitHub Actions — run tests, rebuild Docker images, push to registry on every merge to main

## 7. Deployment & Infrastructure

| Concern            | Recommended Solution                       | Notes                                                          |
|--------------------|--------------------------------------------|----------------------------------------------------------------|
| Model serving      | FastAPI + Uvicorn ASGI server              | Async-capable; auto-generates Swagger docs                     |
| BERT model latency | ONNX Runtime or TorchScript export         | Reduces BERT inference from ~300ms to ~80ms                    |
| Containerisation   | Docker + Docker Compose                    | One Dockerfile per microservice                                |
| Horizontal scaling | Kubernetes HPA (Horizontal Pod Autoscaler) | Scale medicine recommender separately from BERT service        |
| Model versioning   | MLflow or DVC                              | Track experiments, compare model versions, roll back if needed |
| Database           | SQLite → PostgreSQL for                    | SQLite fine for dev; PostgreSQL handles                        |

|                   |                                         |                                                        |
|-------------------|-----------------------------------------|--------------------------------------------------------|
|                   | production                              | concurrent reads better                                |
| Secret management | AWS Secrets Manager / HashiCorp Vault   | Never store API keys or DB credentials in code         |
| Monitoring        | Prometheus + Grafana                    | Track latency, error rates, ADR flag frequency         |
| Data privacy      | HIPAA / GDPR compliance review required | Clinical notes must be de-identified before processing |

---

## 8. Limitations & Future Work

---

### ► Component 1 — Disease Prediction

- The 5,000-note dataset covers only 20 conditions. A real clinical deployment must support hundreds of diagnoses. Priority: expand training data via hospital EHR partnerships.
- The near-perfect F1 (1.00) is likely a reflection of limited dataset variety — notes for the same condition share similar templates. Independent validation on noisy real-world notes is essential before any clinical use.
- Future: Add uncertainty quantification so the API can return a confidence interval alongside its prediction, and flag low-confidence cases for human review.

### ► Component 2 — Medicine Recommendation

- The system uses textual similarity, not pharmacological equivalence. It may recommend medicines with similar descriptions but different active ingredients or drug classes. Pharmacist review of recommendations is required.
- The 806-word vocabulary may not capture subtle therapeutic differences between closely related drugs.
- Future: Integrate structured pharmacological data (drug class, mechanism of action, ATC codes) from DrugBank or ChEMBL to enrich the similarity signal beyond text.

### ► Component 3 — ADR Detection

- SIDER mapping covers only 78 of 502 drugs (15.5%). The remaining 84.5% cannot be cross-verified, limiting the system's ability to distinguish known from novel ADRs.
- The classifier's 25% false-negative rate (57 missed ADR cases out of 232) means roughly one in four genuine adverse reactions is not flagged. For a patient safety system, this is a significant concern.
- Patient review language is informal and subjective. The NER model may miss subtly described symptoms or misinterpret metaphorical expressions.
- Future: Increase SIDER coverage by mapping additional drug names via MedDRA/UMLS codes; add a confidence threshold below which cases are routed to human pharmacovigilance review; retrain on a larger, more diverse review corpus.

---

## 9. Conclusion

---

This report has documented the complete backend pipeline for three AI-powered healthcare components. Each component has been described from raw data ingestion through model training, evaluation, serialisation, and API exposure. The following table summarises the backend artefacts produced by each pipeline:

| Component               | Saved Artefacts                                                                                                                                           | API Endpoint             | Key Metric                                    |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------|-----------------------------------------------|
| Disease Prediction      | patient_model/ (BERT weights),<br>label_encoder.pkl,<br>patient_model.zip                                                                                 | POST /predict-disease    | F1-Score:<br>1.00 (20<br>classes)             |
| Medicine Recommendation | medicine_dict.pkl (9,720 medicines),<br>model.pkl (float16 similarity matrix),<br>new_data.csv                                                            | POST /recommend-medicine | Top-5<br>accuracy:<br>clinically<br>validated |
| ADR Detection           | adr_ner_final_certified/ (NER<br>model), adr_classifier_model.pkl<br>(LR pipeline), adr_database.db<br>(153K SIDER pairs),<br>certified_drug_mappings.csv | POST /detect-adr         | NER F1:<br>99.5%,<br>AUC:<br>85.2%            |

All three models follow software engineering best practices: clean data/model separation, reproducible training scripts, versioned artefacts, and structured API contracts. The platform is ready for containerisation and staged clinical validation, beginning with a controlled pilot study under medical supervision before any live patient-facing deployment.

Recommended Next Steps

1. Containerise all three services with Docker and deploy to a staging environment.
2. Conduct clinical validation of Disease Prediction with a hospital partner using real de-identified EHR notes.
3. Enrich Medicine Recommendation with structured pharmacological metadata from DrugBank.
4. Expand SIDER drug-name mapping to cover more of the 502 dataset drugs.
5. Implement a human-review queue for ADR predictions with probability in the 40–60% borderline range.
6. Obtain ethics board approval and HIPAA/GDPR compliance certification before any clinical pilot.